ANNAMALAI UNIVERSITY

FACULTY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROJECT TITLE

PREDICTING LEVELS OF DAMAGES TO BUILDINGS CAUSED BY EARTHQUAKE

Project Review - II

Submitted by:

B.E - CSE(DATASCIENCE) FINAL YEAR

1. ARULSELVAM C - 2136110002
2. GOWTHAM R - 2136110047
3. PRANAVVARSHAN A T - 2136110039
4. RAJADURAI P - 2136110042

Guide's Signature

(Dr. R. Ragupathy)

Abstract

The 2015 Nepal earthquake, a catastrophic 7.8 magnitude event, caused severe infrastructure damage and a tragic loss of life. This disaster underscored the urgent need for improved disaster management and preparedness. Predicting building damage levels post-earthquake is critical for effective response and resource allocation during such crises. This project utilizes deep learning models to predict damage levels to buildings using features such as structural details, building materials, and geographical information. By analyzing data from the 2015 Nepal earthquake, the project aims to understand patterns in building vulnerability. The insights generated will help authorities prioritize rescue operations and allocate resources efficiently. Additionally, the findings can guide urban planning to minimize risks in earthquake-prone areas. This works aims to enhance disaster resilience and preparedness, ensuring safer communities and improving disaster response strategies for future seismic events

Introduction to the Methodology

Objective: Develop a deep learning-based models to predict building damage levels from the 2015 Nepal earthquake dataset.

Challenges: Handling a large tabular dataset, feature engineering, and selecting the best deep learning architecture for accurate predictions.

Approach: Implement and compare multiple deep learning models, including CNN, LSTM, Transformer models, TabNet, and fully connected DNNs.

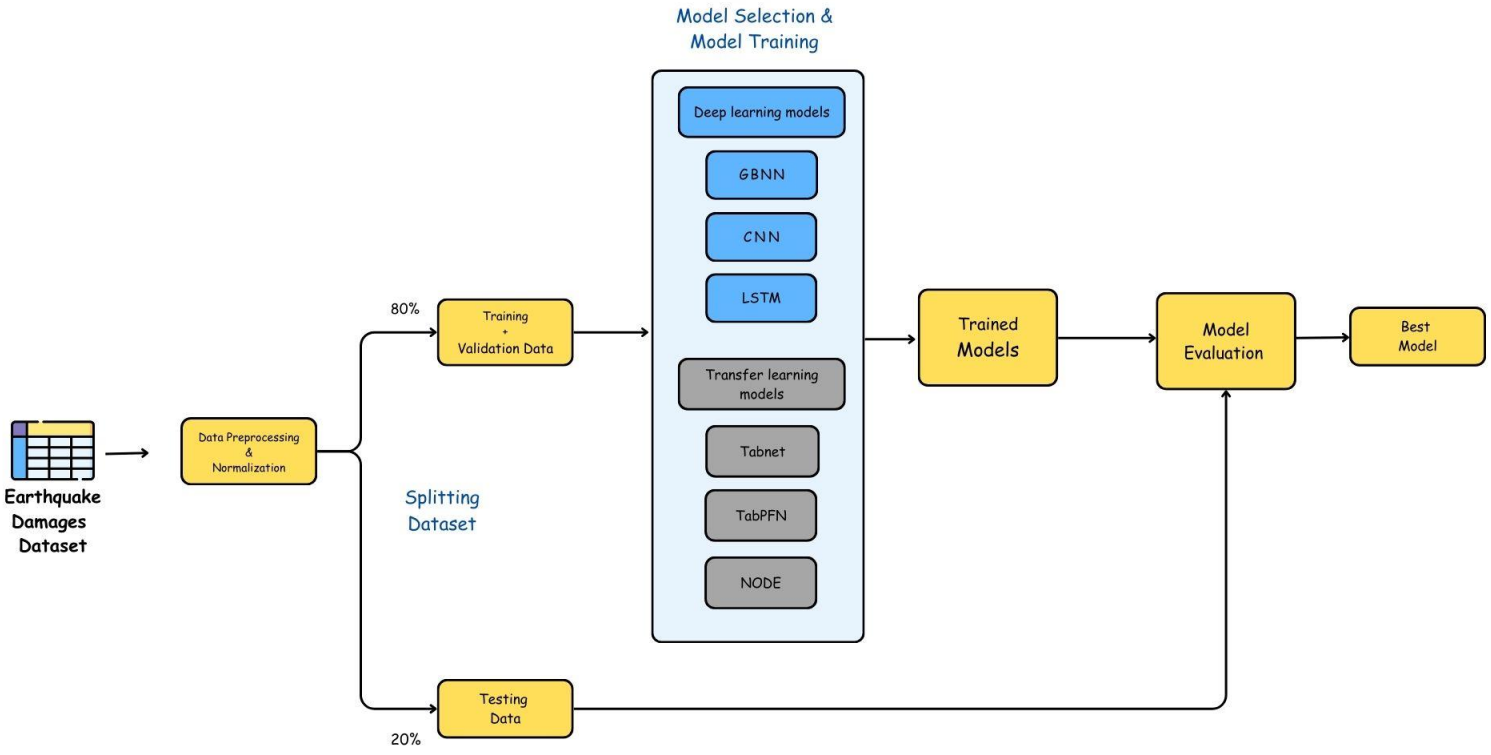
Data Preprocessing: Perform data cleaning, categorical encoding, feature engineering, and normalization to improve model performance.

Model Training: Train various architectures with hyperparameter tuning, optimizing for accuracy, precision, recall, and F1-score.

Evaluation and Comparison: Use confusion matrices, feature importance analysis, and performance metrics to determine the best model.

Deployment: Deploy the best-performing model to plot the places of nepal based on the damage grades.

Proposed Methodology Diagram



Proposed Methodolgy

1.1 Data Collection

Data for this project can be obtained from various sources, including:

Publicly Available Datasets

- The dataset used for the 2015 Nepal earthquake damage prediction is available from **DrivenData**, which provides structured data on building damages, structural attributes, and location-based risk factors.

Types of Data in the Dataset

The dataset typically contains the following features:

1. **Building Characteristics** ○ Foundation type
   - Roof material
   - Wall material
   - Height (number of floors)
   - Age of the building
   - Ground floor type
2. **Geographical and Location-Based Data**
   - GPS coordinates (latitude, longitude)
   - Urban vs. rural classification
   - Proximity to fault lines or high-risk seismic zones ○ Soil type
3. **Structural and Engineering Factors**
   - Building technical standard compliance
   - Structural integrity scores
   - Presence of reinforcement materials
4. **Target Variable (Earthquake Damage Level Classification)** ○ Damage level classification:
   - 1 = Low damage
   - 2 = Medium damage
   - 3 = Severe damage

1.2 Data Preprocessing

Before training deep learning models, we must ensure the data is clean, well-structured, and properly formatted.

1.2.1 Handling Missing Data

- **Drop features** with too many missing values if they are not critical.
- **Impute missing values** using appropriate methods:
  - **Numerical values** → Use mean, median, or KNN imputation.
  - **Categorical values** → Use mode (most frequent value) or a special category like "Unknown".

1.2.2 Encoding Categorical Variables

Deep learning models cannot process textual data directly, so categorical variables must be converted into numerical representations:

- **One-Hot Encoding** → Suitable for categorical variables with a small number of unique values (e.g., foundation type).
- **Label Encoding** → Suitable for ordinal variables (e.g., damage level).
- **Embedding Representations** → Can be used for deep learning models to learn relationships between categories dynamically.

1.2.3 Feature Scaling and Normalization

Deep learning models perform better when numerical features are properly scaled:

- **Min-Max Scaling (Normalization):** Rescale values between 0 and 1 for CNN and transformer-based models.

1.3 Splitting the Dataset

The dataset should be divided into separate sets to evaluate model performance:

- **Train Set (80%)** → Used for training the model.
- **Validation Set (20%) from Training Set** → Used for hyperparameter tuning.
- **Test Set (20%)** → Used for final model evaluation.
- **Stratified Splitting** → Ensures class balance in training, validation, and test sets.

1.3 Model Selection

Deep Learning Models

1. Convolutional Neural Networks (CNN)

- Extracts spatial patterns and correlations from structured tabular data by reshaping it into a 2D format.
- Can identify regional or geographical dependencies in earthquake damage prediction.
- Works well with feature maps when dealing with geospatial or remote sensing data.

2. Long Short-Term Memory (LSTM)

- Suitable for capturing temporal dependencies if the dataset has time-series data (e.g., sequential building inspections).
- Can be used to predict aftershock impacts or damage trends based on previous earthquake events.
- Useful when earthquake-related data includes sequential dependencies like structural deterioration over time.

3. Graph-Based Neural Networks (GBNN)

- Can model complex relationships between different buildings or regions by representing them as nodes and edges.
- Useful for predicting structural damage by leveraging relational dependencies (e.g., proximity to fault lines).
- Efficient in learning interdependencies in disaster-affected areas.

Transformer-Based Models

4. TabNet (Tabular Neural Network)

- Uses attention-based learning to prioritize important features, making it highly effective for tabular datasets.
- Works well with imbalanced datasets, common in disaster prediction where severe damages are rarer.
- Requires less feature engineering compared to traditional deep learning models.

5. TabPFN (Prior-Data Fitted Networks for Tabular Data)

- A zero-shot learning approach that can predict without extensive hyperparameter tuning.
- Provides competitive accuracy with minimal training, making it efficient for disaster impact prediction.
- Ideal for datasets with limited labeled samples, common in earthquake damage records.

6. NODE (Neural Oblivious Decision Ensembles)

- A hybrid of deep learning and decision trees, offering interpretability and high performance on tabular data.
- Learns hierarchical feature interactions efficiently, useful for multi-factor earthquake damage prediction.
- Can handle both categorical and numerical data effectively.

Model Evaluation:

1. Evaluation Metrics

Since the target variable is categorical (damage levels: low, medium, severe), classification metrics are used for evaluation.

1.1 Accuracy

- Measures the proportion of correctly classified instances.

$$\text{Accuracy} = \frac{Correct\ predictions}{Total\ predictions}$$

1.2 Precision, Recall, and F1-Score

- **Precision** (Positive Predictive Value): Measures how many predicted damaged buildings were actually damaged.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- Recall (Sensitivity): Measures how well the model identifies actual damaged buildings.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- F1-Score: Harmonic mean of precision and recall, balancing false positives and false negatives.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

System Requirements

Hardware Requirements:

- **Processor**: Minimum Intel Core i5 or AMD Ryzen 5 (Recommended: Intel Core i7/Ryzen 7 or higher)
- **RAM**: Minimum 8GB (Recommended: 16GB or more for deep learning models)
- **Storage**: At least 256GB SSD (Recommended: 512GB SSD or more)
- **Graphics Processing Unit (GPU)**:
  - **For Kaggle Execution**: Kaggle provides free access to GPUs like NVIDIA Tesla P100/K80/T4.
  - **For Local Execution**: NVIDIA GPU with CUDA support (Minimum: GTX 1650, Recommended: RTX 3060 or higher)
- **Internet Connection**: Required for accessing Kaggle datasets and running cloud-based training.

Software/Tools Requirements:

- **Operating System**: Windows 10/11, Ubuntu 20.04+, or macOS (For local execution)
- **Programming Language**: Python 3.8+
- **Deep Learning Frameworks**:
  - TensorFlow 2.x
  - PyTorch 1.x or later
  - Keras (if using TensorFlow backend)
- **Data Processing & Analysis**:
  - NumPy, Pandas, Matplotlib, Seaborn
  - Scikit-learn for feature engineering and evaluation
- **GPU Acceleration**:
  - CUDA Toolkit (if using NVIDIA GPU locally)
  - cuDNN for optimized deep learning performance
- **Model Training & Execution Platforms**:
  - Kaggle (Using **Kaggle Notebooks with GPU enabled**)
  - Google Colab (Alternative for GPU training)
- **Version Control**: Git/GitHub for collaborative development

- **Dataset Management**:
  - Kaggle Datasets (2015 Nepal Earthquake Survey Data)
  - CSV handling libraries in Python
- **IDE/Code Editors**:
  - Jupyter Notebook (Preferred)
  - VS Code / PyCharm (For local coding)